

**VIETNAM NATIONAL UNIVERSITY – HOCHIMINH CITY
INTERNATIONAL UNIVERSITY
SCHOOL OF COMPUTER SCIENCE AND ENGINEERING**



ALGORITHM AND DATA STRUCTURES

IT013IU

Topic: MINESWEEPER

Submitted by

Huynh Dat Minh Tam – ITITI21305

Mai Duc Thien – ITITI21319

. Course Instructor: Mr T.T.Tin

Contents

CHAPTER 1 INTRODUCTION.....	2
1. Course Abstract	2
2. Overview	2
CHAPTER 2 GAME INSTRUCTION AND FEATURE	3
1. Objective	3
2. Game rules	3
CHAPTER 3 GAME DETAILS	4
1. Data structure	4
2. Algorithms and Time Complexity	5
3. Game Design and Implements	11
CHAPTER 4 FUTURE WORK	22
CHAPTER 5 CONCLUSION	22

CHAPTER 1 INTRODUCTION

1. Course Abstract

Data structures and algorithms are essential in computer science for efficient data management and problem-solving. Data structures like arrays, linked lists, stacks, queues, trees, graphs, and hash tables optimize data access and modification. Algorithms, such as sorting (quicksort, merge sort) and searching (binary search), use these structures to perform tasks efficiently. Mastery of these concepts is crucial for developing scalable and effective software, impacting areas like databases, networking, and artificial intelligence.

2. Overview

Minesweeper is a classic single-player puzzle game where the objective is to clear a grid of covered tiles without clicking on any hidden mines. Players uncover safe tiles using numbered clues, which reveal how many mines are present in the surrounding adjacent squares.

CHAPTER 2 GAME INSTRUCTION AND FEATURE

1. Objective

The objective of Minesweeper is to clear a hidden board of tiles without clicking on any concealed mines. Doing this by clicking safe squares to reveal numeric clues and flagging tiles you suspect are hazardous.

2. Game rules

Start the game:

- Click any square to begin. The first click is always safe and will generate the numbers you need to start playing.

Understand the numbers:

- When you click a square, it will either be blank or display a number (from 1 to 8). This number tells you exactly how many mines are hidden in the 8 squares immediately surrounding that numbered square (horizontally, vertically, and diagonally).
- Use the numbers to figure out which surrounding squares are safe to click and which are mines.
- If you figure out where a mine is, right-click the square (or tap and hold on mobile) to place a flag. This prevents you from accidentally clicking it.

Win Condition:

- You win the game once all safe, non-mine squares have been successfully uncovered.

CHAPTER 3 GAME DETAILS

1. Data structure

- Arrays
 - To create the game board, I used a 2D array, which effectively represents the grid and the distribution of mines. This structure allows for efficient access and manipulation of the grid's cells, ensuring smooth gameplay and accurate mine placement.

```
public class Game {
    private final App a;

    public Cell[][] gBoard = new Cell[0][0];
    public GameState gGame = new GameState();
    public Level gLevel = new Level();

    public Game(App a) {
        this.a = a;
        this.gLevel.bestTime = a.store.getInt("bestTimeEasy");
    }
}
```

- Stacks
 - I implemented an undo feature using a stack data structure. This allows players to revert their moves, enhancing the user experience by providing a way to correct mistakes and explore different strategies without restarting the game.
 - A stack can be represented using an array, utilizing the pop and push methods.

```
public class Features {
    private final App a;

    public Deque<Object> gMoves = new ArrayDeque<>();
    public List<Cell> gNrMoves = new ArrayList<>();
    public boolean gHintOn = false;
}
```

2. Algorithms and Time Complexity

2.1. Structure of the board game

- gBoard: where to store the cells.
- gGame: define all information of this game.
- gLevel: define the level of this game, “Easy” is a default.

```
public Cell[][] gBoard = new Cell[0][0];
public GameState gGame = new GameState();
public Level gLevel = new Level();
```

2.2. Initialize the game

```
public void init() {
    a.gPlayer.setText(App.NORMAL);
    gBoard = buildBoard();
    a.render.renderBoard(gBoard);

    int size = gLevel.size;

    a.gLives.setText(size == 4 ? "♥" : "♥♥♥♥");
    a.gHints.setText(size == 4 ? "Hint: 💡" : "Hints: 💡💡💡");
    a.gSafe.setText(size == 4 ? "🔍" : size == 8 ? "🔍🔍" : "🔍🔍🔍");

    if (gLevel.bestTime == 0) return;
    a.timer.renderBestTime();
}
```

- init() is a beginning function of this game.
- The logic of this function:
- buildBoard() => renderBoard() => show the detail informations.

2.3. Build the board

```
public Cell[][] buildBoard() {
    Cell[][] board = new Cell[gLevel.size][gLevel.size];
    for (int i = 0; i < gLevel.size; i++) {
        for (int j = 0; j < gLevel.size; j++) {
            board[i][j] = new Cell(i, j);
        }
    }
    return board;
}
```

- The time complexity: $O(n^2)$.
- The outer loop will run each row of this board.
- In each row, it will first create an array, then it continues to run through all the subsequent columns. In each column, it will define that position as a cell and add the following information for that cell: minesAroundCount, isShown, isMarked, isMine, and position.

2.4. Render the board

```
public void renderBoard(Cell[][] board) {
    int size = board.length;
    cells = new JLabel[size][size];
    pressed = new boolean[size][size];
    safe = new boolean[size][size];

    a.boardPanel.removeAll();
    a.boardPanel.setLayout(new GridLayout(size, size));

    for (int i = 0; i < size; i++) {
        for (int j = 0; j < size; j++) {
            final int fi = i;
            final int fj = j;

```

- The time complexity: $O(n^2)$.
- It will loop each row; in each row it will loop each column.

2.5. Render the best time

```
public void renderBestTime() {
    int bestTime = a.game.gLevel.bestTime;
    if (bestTime < 60) {
        a.gBestTime.setText("🕒 " + bestTime + " seconds");
    } else if (bestTime > 60) {
        double mins = bestTime / 60.0;
        a.gBestTime.setText("🕒 " + String.format("%.2f", mins) + " minutes");
    }
}
```

- Time complexity: $O(1)$.
- It will get the value of bestTime stored in gLevel.
- If this time greater than 60, it will show the time with minutes format.
- If this time is less than, it will display the time in seconds.

2.6. Restart function

```
public void restart() {
    a.timer.stop();
    gGame.isRun = false;
    gGame.isFirst = true;
    gGame.shownCount = 0;
    gGame.markedCount = 0;
    gGame.secsPassed = 0;
    a.gTimer.setText("00:00");
    gLevel.hints = gLevel.size == 4 ? 1 : 3;
    gLevel.lives = gLevel.size == 4 ? 1 : 3;
    gLevel.safeClicks = 1;
    a.feature.gMoves.clear();
    a.feature.gNrMoves = new ArrayList<>();
    a.gBestTime.setText("");
    init();
}
```

- Time complexity is $O(1)$.
- Stopping the timer.
- Change all information of this game to default.

- Display the timer to default.
- Display the features to default.
- Empty the stack.
- Remove the best time.

2.7. Undo feature

```
public void undo() {
    if (!a.game.gGame.isRun) return;
    if (gMoves.isEmpty()) return;

    Object move = gMoves.pop();

    if (move instanceof List) {
        for (Cell c : (List<Cell>) move) {
            reverseMove(c);
        }
    } else {
        reverseMove((Cell) move);
    }
}

public void reverseMove(Cell cell) {
    cell.isShown = false;

    if (cell.isMine) {
        a.game.gLevel.lives++;
        a.gLives.setText(a.gLives.getText().equals("❤️❤️❤️") ? "❤️❤️❤️" : "❤️❤️❤️");
    }

    a.render.removePressed(cell.i, cell.j);
    a.render.renderCell(cell.i, cell.j, "");
}
```

- Time complexity is $O(n)$
- First, if the previous move change status for more than one cell, run the loop for each cell change and call reverseMove() function for each cell.
- Second, if the previous move just change the status of one cell, call reverseMove() function for this cell.
- The reverseMove() function change the status isShown to false, if this cell is mine increase the lives and display it to player. Remove the class 'pressed' and render this cell to empty

2.8. Show safe cell feature

```
public void showSafeCell() {
    if (!a.game.gGame.isRun) return;
    if (a.game.gLevel.safeClicks == 0) return;

    a.game.gLevel.safeClicks--;
    int remains = a.game.gLevel.safeClicks;

    a.gSafe.setText(remains == 1 ? "🕒" : remains == 2 ? "🕒🕒" : "X");

    List<int[]> safeCells = new ArrayList<>();
    Cell[][] board = a.game.gBoard;
    for (int i = 0; i < board.length; i++) {
        for (int j = 0; j < board[0].length; j++) {
            Cell cell = board[i][j];
            if (cell.isMine || cell.isShown) continue;
            safeCells.add(new int[] { i, j });
        }
    }
    if (safeCells.isEmpty()) return;

    int[] randomIndex = safeCells.get(a.util.getRandomInt(0, safeCells.size()));
    final int si = randomIndex[0];
    final int sj = randomIndex[1];

    a.render.addSafe(si, sj);

    javax.swing.Timer t = new javax.swing.Timer(2000,
        e -> a.render.removeSafe(si, sj));
    t.setRepeats(false);
    t.start();
}
```

- Time complexity is $O(n^2)$.
- It will scan all the cell in this board, if the cell is mine or shown, continuously; if the cell is not mine or shown add this position to the safeCells array.
- Finally, random a position in safeCells and display it for the player, and setTimeout 2 seconds to hide this feature.

2.9. Show hint feature

```
public void undo() {
    if (!a.game.gGame.isRun) return;
    if (gMoves.isEmpty()) return;

    Object move = gMoves.pop();

    if (move instanceof List) {
        for (Cell c : (List<Cell>) move) {
            reverseMove(c);
        }
    } else {
        reverseMove((Cell) move);
    }
}

public void reverseMove(Cell cell) {
    cell.isShown = false;

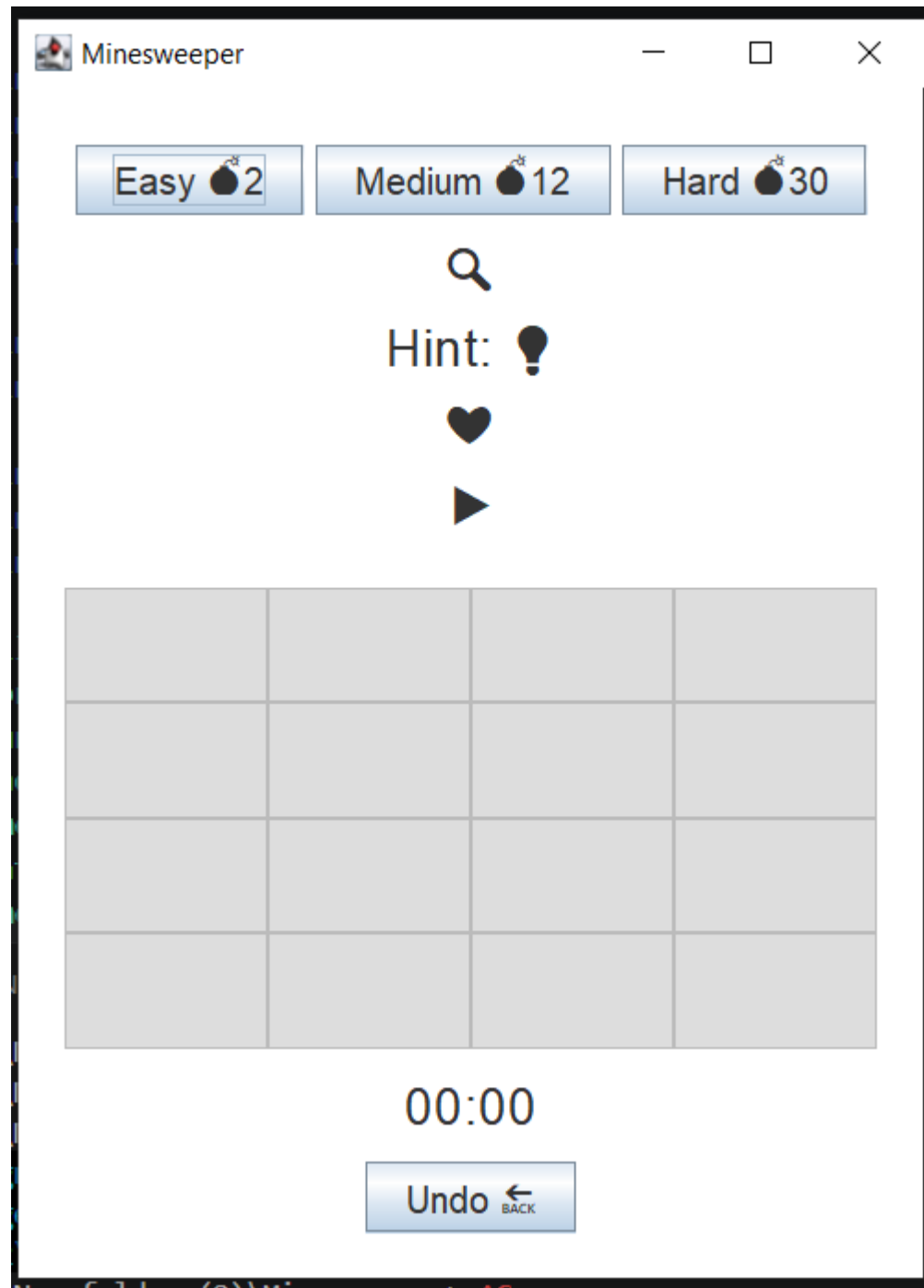
    if (cell.isMine) {
        a.game.gLevel.lives++;
        a.gLives.setText(a.gLives.getText().equals("❤️❤️❤️") ? "❤️❤️❤️" : "❤️❤️❤️");
    }

    a.render.removePressed(cell.i, cell.j);
    a.render.renderCell(cell.i, cell.j, "");
}
```

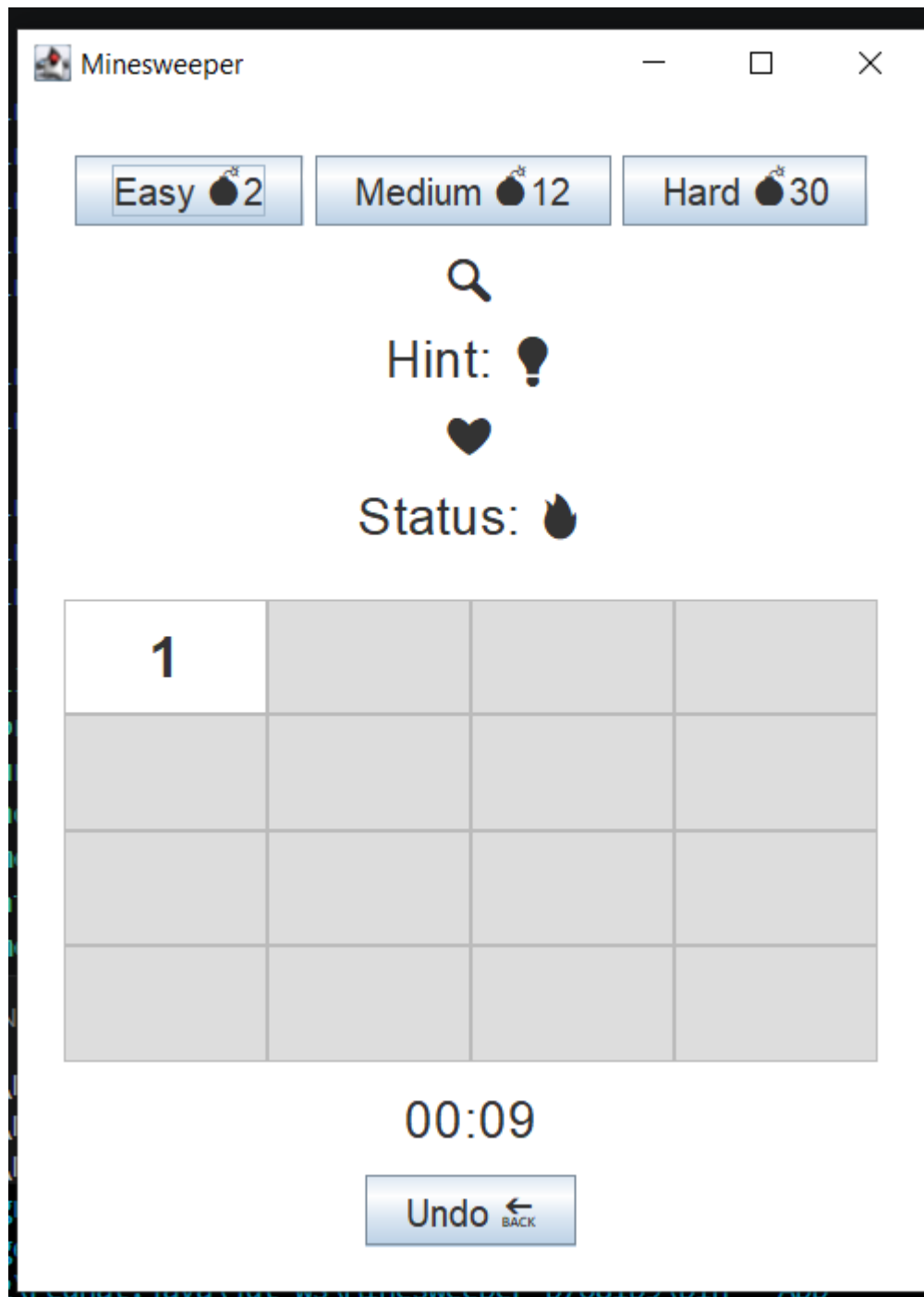
- Time complexity: $O(1)$.
- This is the algorithm to update the hint remaining for this game and display it for player.

3. Game Design and Implements

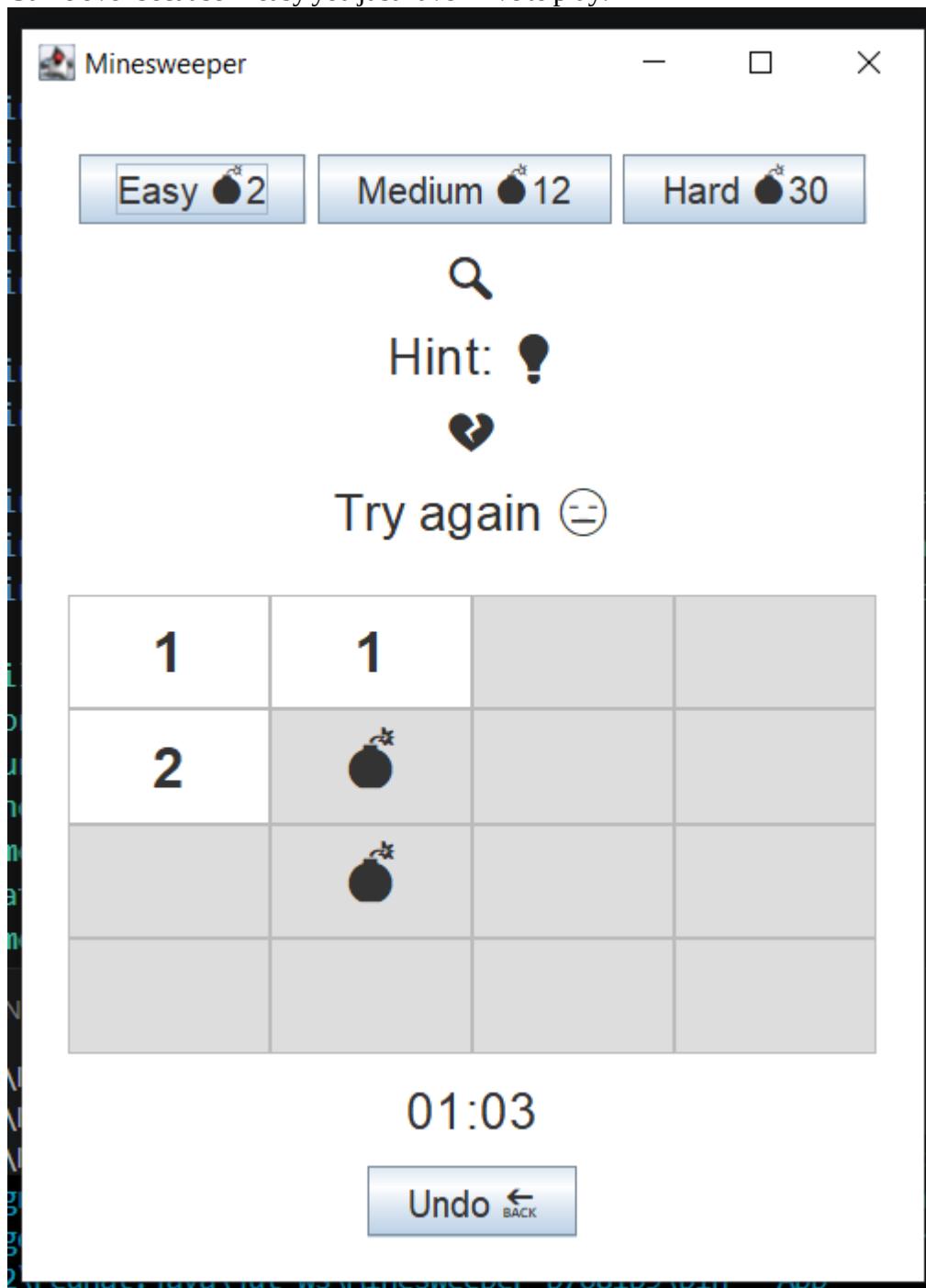
- The initialization of this game:



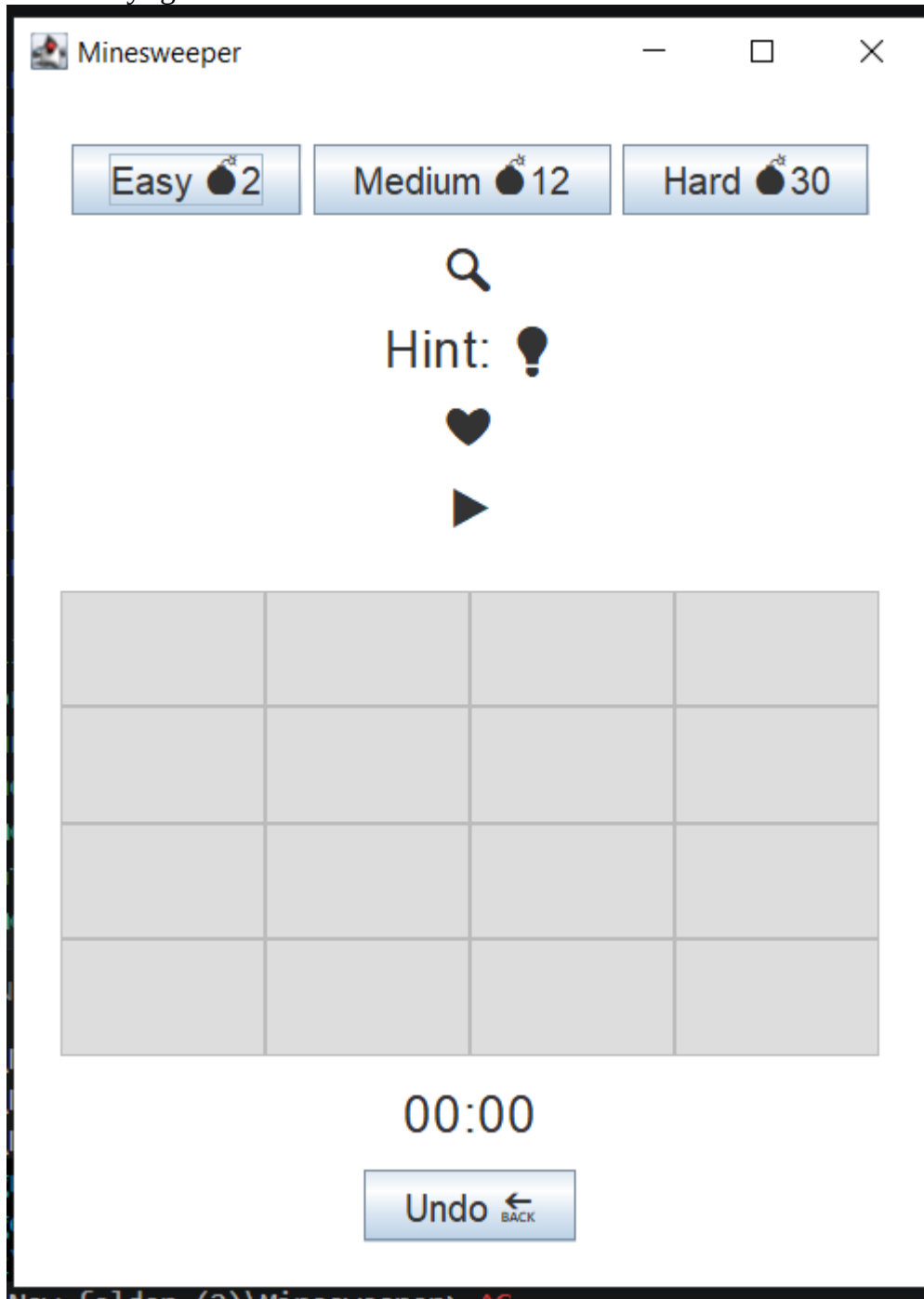
- Start the game:



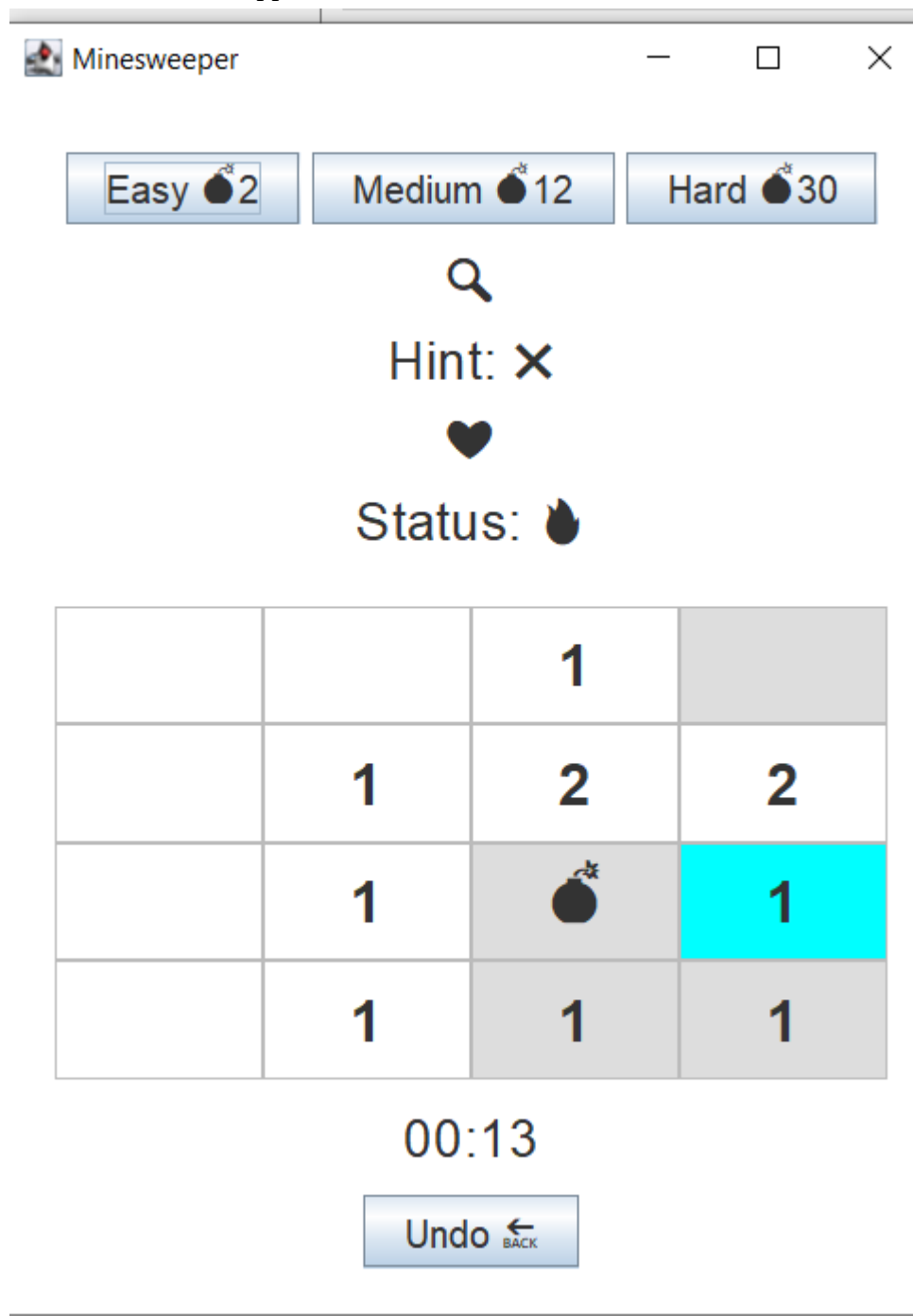
- Game over because in easy you just have 1 live to play:



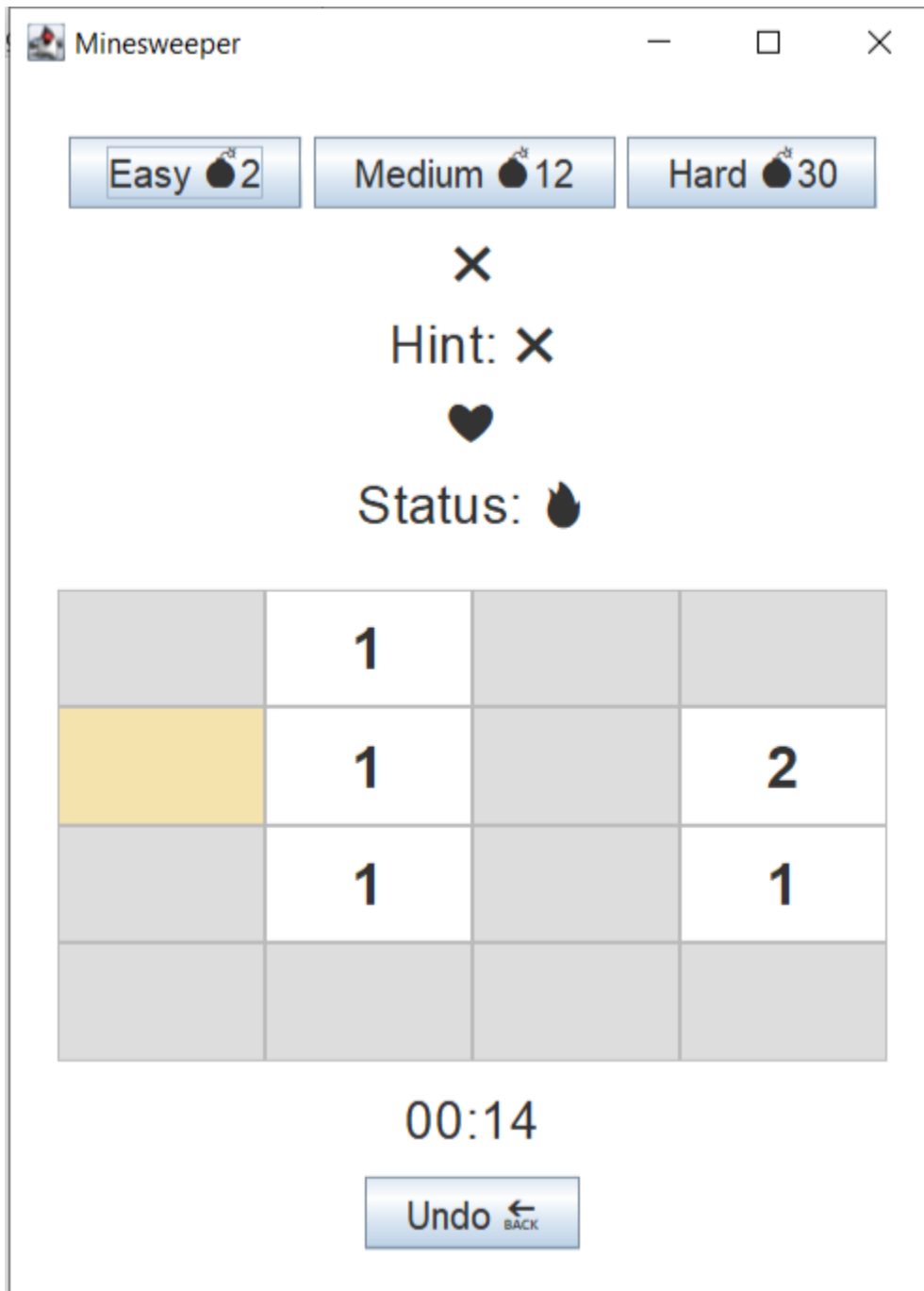
- Click “Try again” to continue:



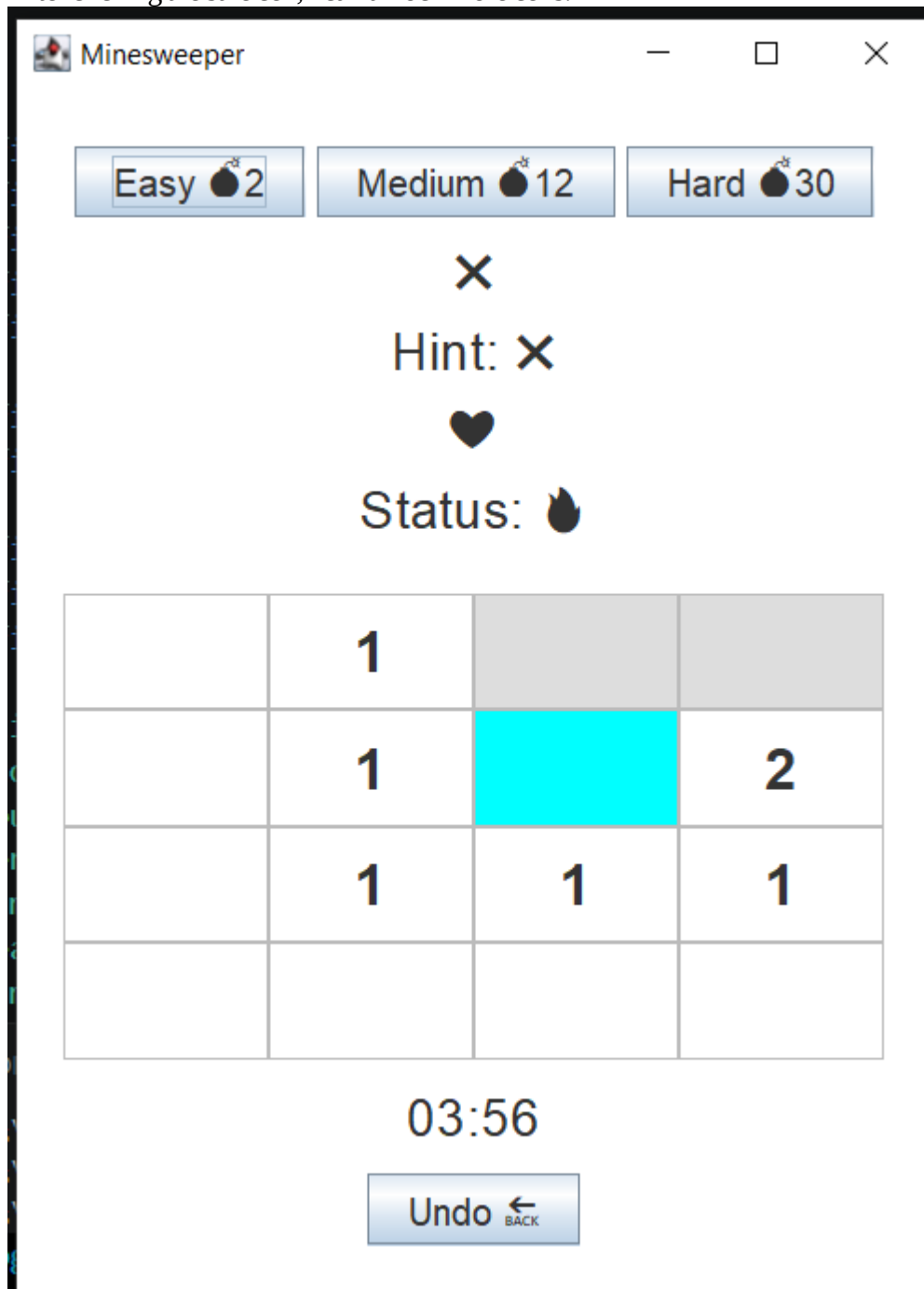
- When I use “Hint”, the total hints will be decrease by 1. This feature will show all cell in around 3x3 then disappear in 2 seconds:



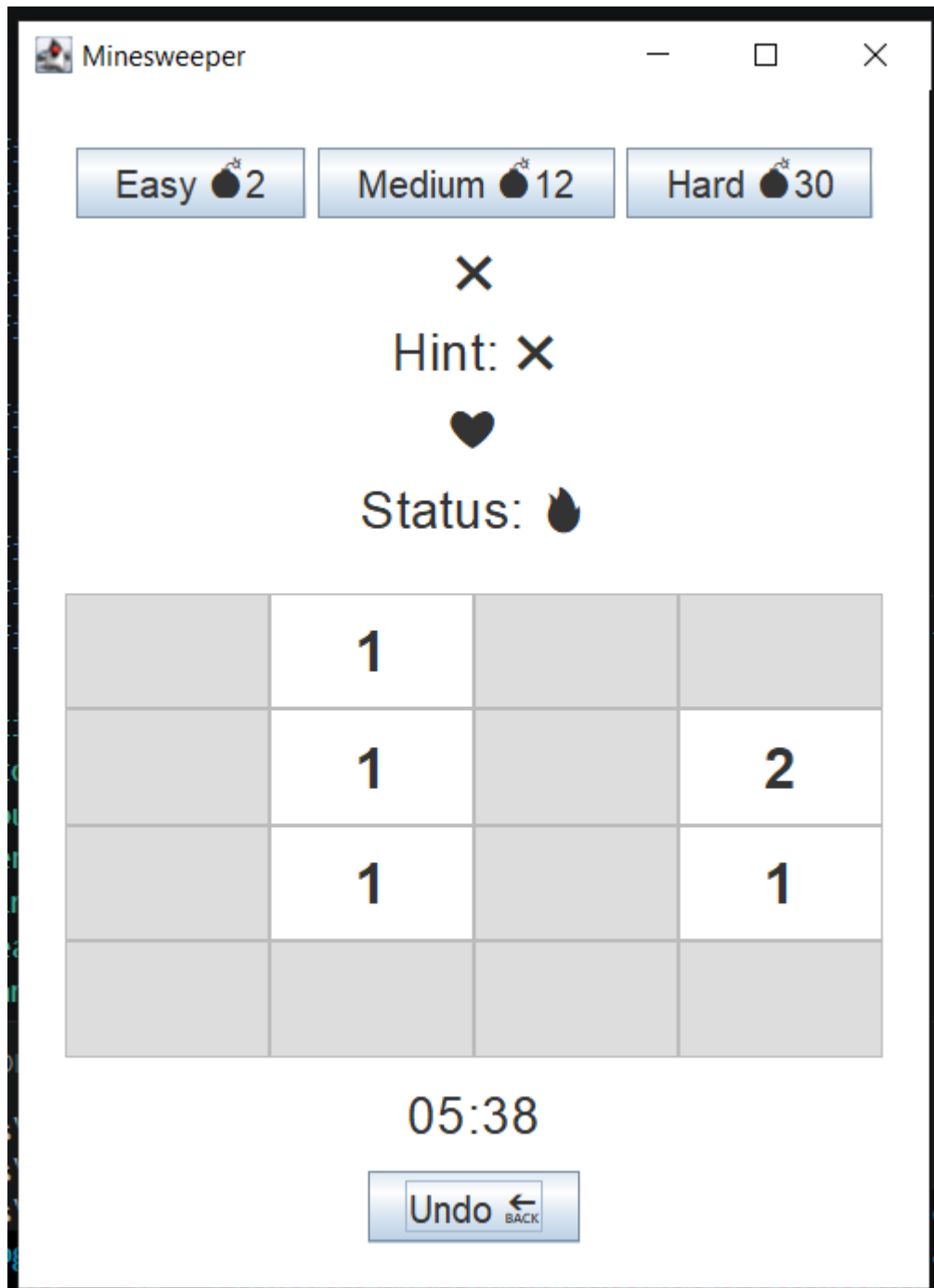
- When I use “Safe Click”, it will find all safe cell and avoid cell has mine, then it will random to show 1 cell for player:



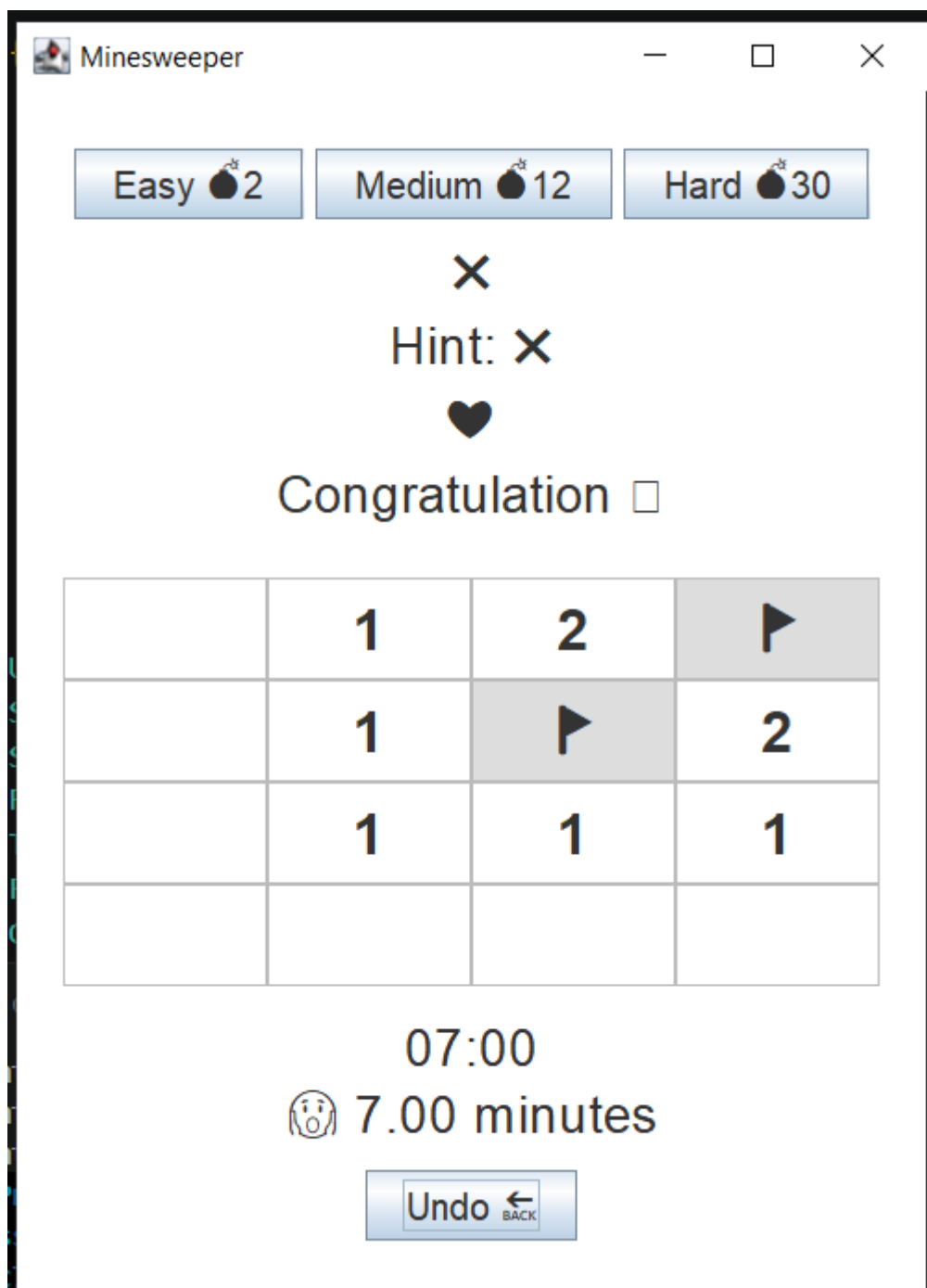
- After clicking the safe cell, I can unlock more cells:



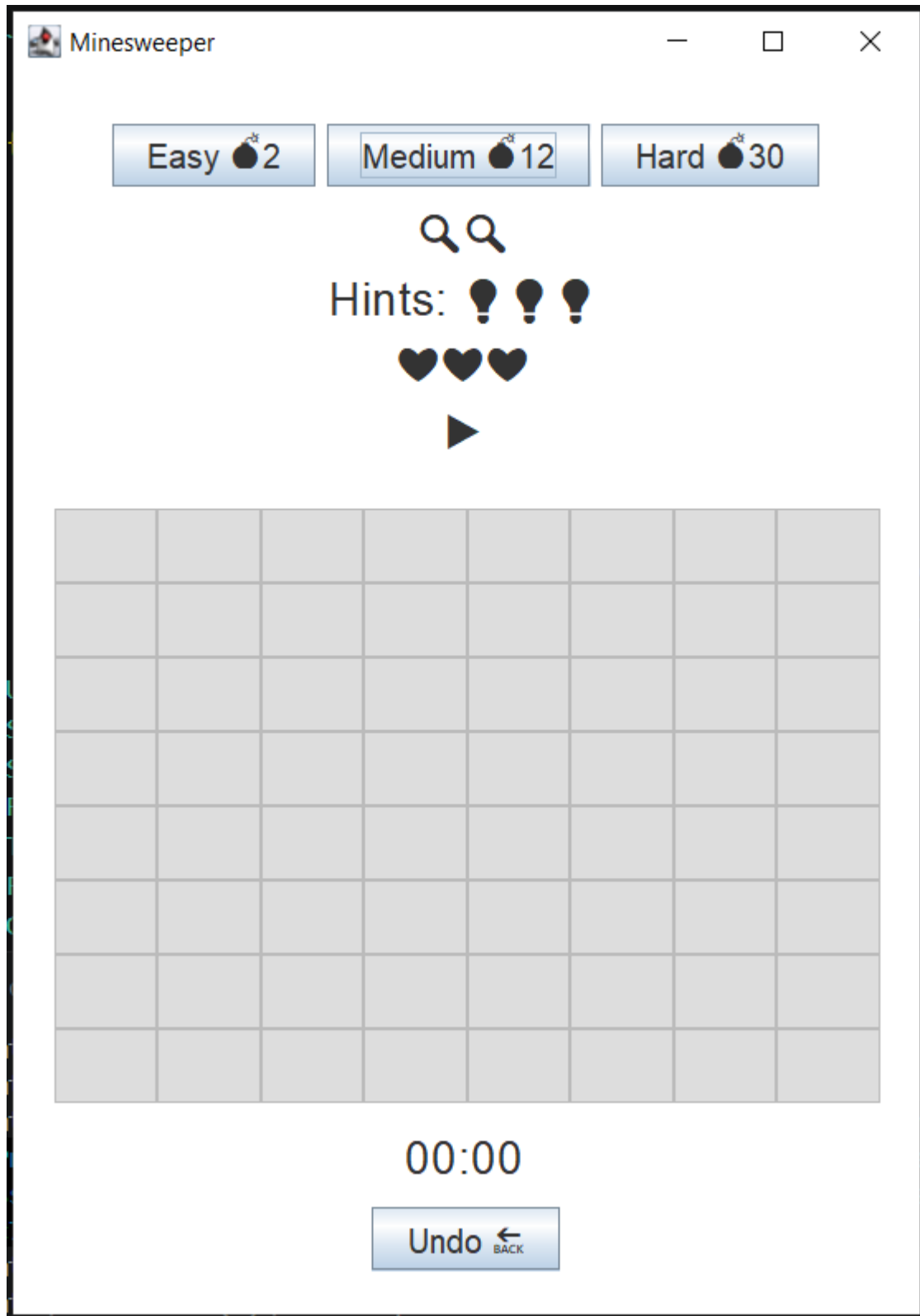
- I also can click “Undo” to comeback the previous action:



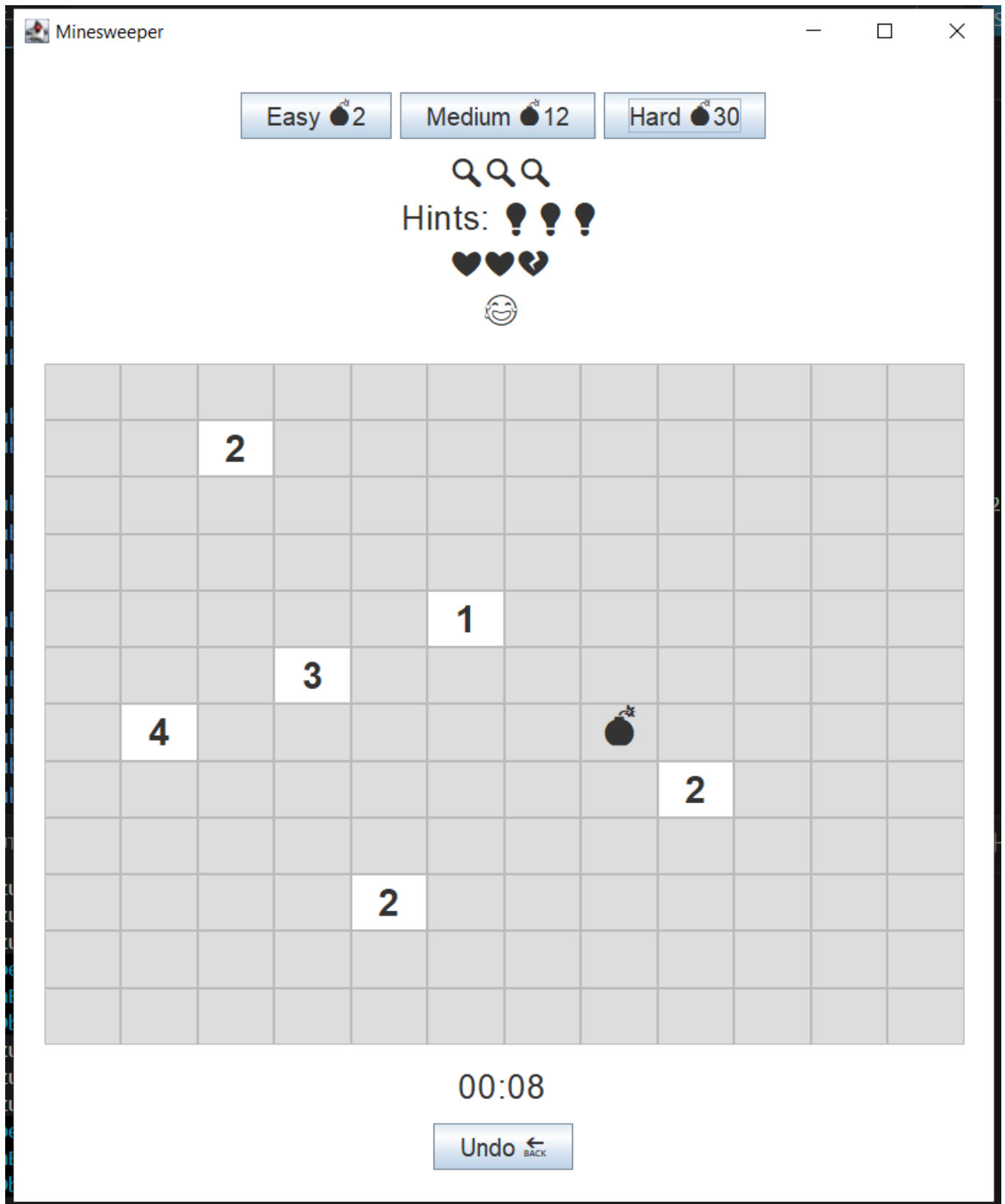
- If I unlock all safe cell and marked true mine cells, I will receive a message that I win that game, then compare with the previous best time if it less than, updating it:



- With the same of algorithms, in medium level I have the table 8x8 and 12 mines.
Increase the time to use “Hints” and “Safe cell”:



- And this is the hard mode with 30 mines with table size 12x12:



CHAPTER 4 FUTURE WORK

Future improvements to the Minesweeper game could include enhanced graphics and more vibrant and smooth visuals, optimization for mobile devices, and new game modes such as timed challenges. Integrating social networks with in-game leaderboards and achievements could boost player interaction. Additionally, the addition of sound effects and data analysis to optimize the user experience are promising improvements. These developments would make Minesweeper a more engaging and exciting game.

CHAPTER 5 CONCLUSION

In conclusion, the development of the Minesweeper game as part of the Algorithms & Data Structures course has been an enlightening and rewarding experience. This project provided the theoretical to create the game covered in the course, particularly in the areas of algorithm design and data structure implementation.